

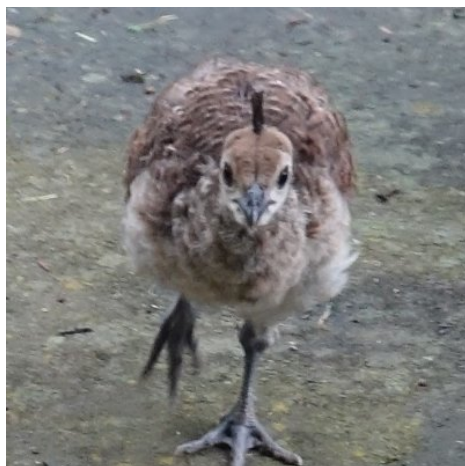
ここ半年くらいで AI に作らせた R 用ツール

arf console, rd2qmd, jgd, ...

@eitsupi

Tokyo.R 121 — 2026-07-25

自己紹介



@eitsupi

- 本業：手料理サブスク Tsuklio のデータ周り
- 副業：Columnar 社で Apache Arrow ADBC 周りの開発
- Excel が嫌になり R を使い始めて 7 年
- Rocker プロジェクト (Shell、R、Docker)
- Polars R パッケージ (R、Rust)
- 最近は arf の人、後述

最近やったこと

2025

r-polars の開発

Japan.R 2025 で発表

2026

エージェントイックコーディングに移行

これまで手を出せなかったツール群を作成

このスライドも Codex CLI に Typst で作らせた

最近作ったツール

arf

R 端末

radian の置き換え (Rust)

rd2qmd

Rd → Markdown 変換

pkgdown の置き換え (Rust)

jgd

R グラフィックデバイス

httpgd の置き換え (C、TypeScript、Go、Rust)

ツール間のつながり

arf、rd2qmd、jgd はそれぞれ無関係に始めたが、意外とそれぞれが関連して活きた

rd2qmd → arf の内蔵 Help Browser

arf の IPC ・ Headless → jgd の AI エージェントによるバグ潰し ・ E2E テスト

arf

Rust 製の R コンソール

radian を置き換えたい

- radian を何年も使っていたが、Python 製なのが嫌だった
 - インストール面倒 (uv 使えば今は楽ですが)
 - Windows 上の文字コード問題
- 「Ark と Nushell を組み合わせればできるのでは？」というところから開始、
なので Rust 製

インストール

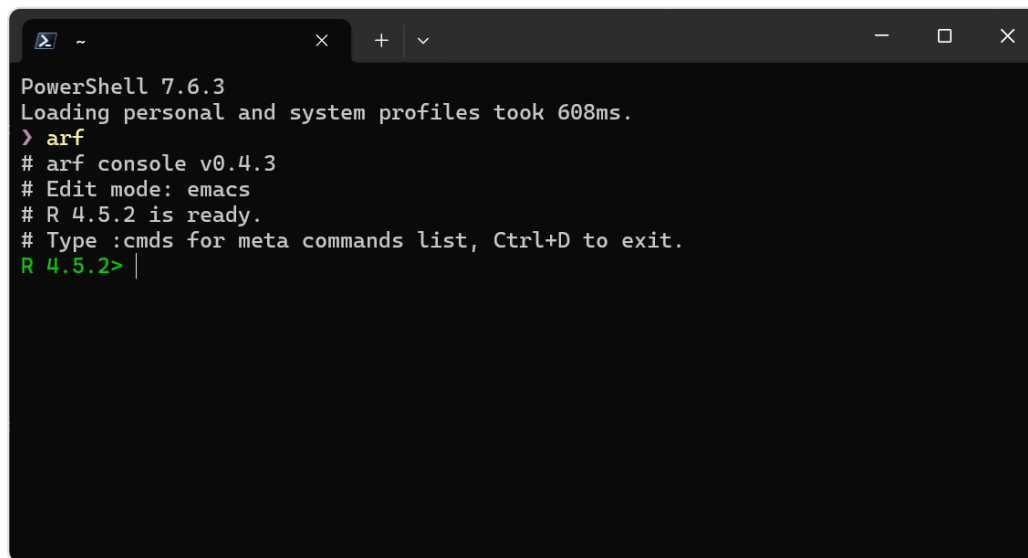
GitHub からダウンロードするか
パッケージマネージャーで。

Windows

```
winget install --id  
eitsupi.arf
```

macOS / Linux

```
brew install arf
```

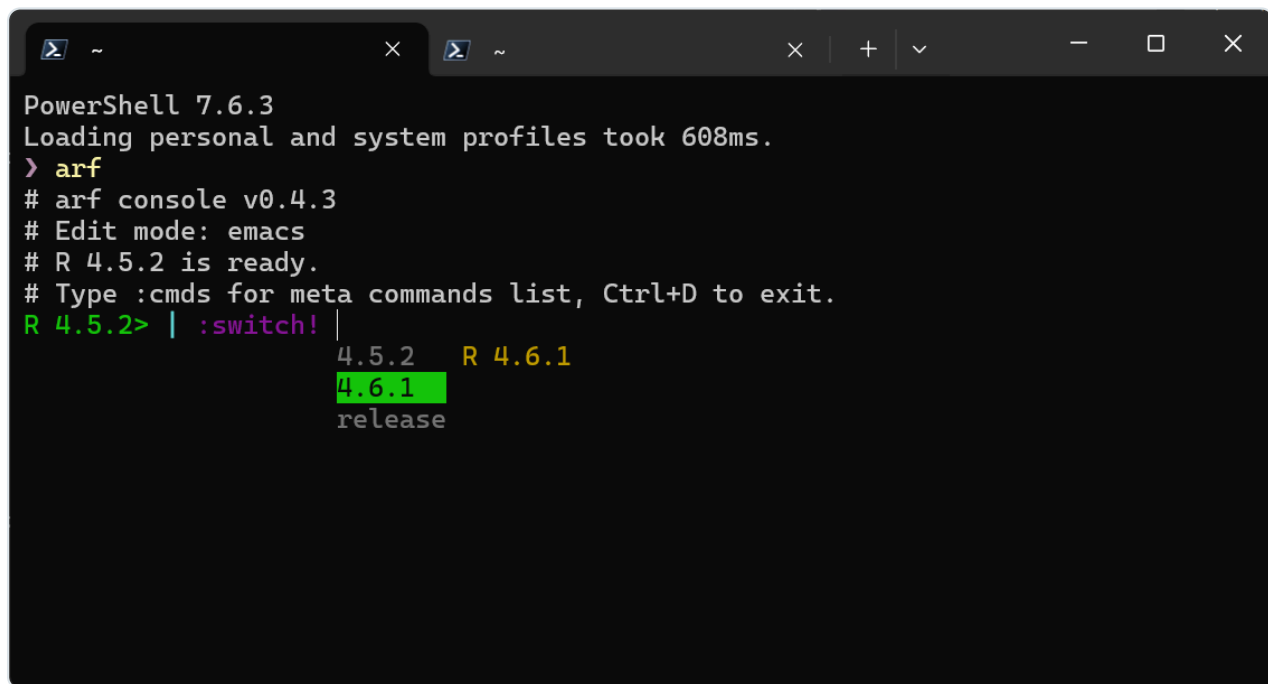


```
PowerShell 7.6.3  
Loading personal and system profiles took 608ms.  
> arf  
# arf console v0.4.3  
# Edit mode: emacs  
# R 4.5.2 is ready.  
# Type :cmds for meta commands list, Ctrl+D to exit.  
R 4.5.2> |
```


機能紹介

arf

R バージョン切り替え



```
PowerShell 7.6.3
Loading personal and system profiles took 608ms.
> arf
# arf console v0.4.3
# Edit mode: emacs
# R 4.5.2 is ready.
# Type :cmds for meta commands list, Ctrl+D to exit.
R 4.5.2> | :switch! |
               4.5.2    R 4.6.1
               4.6.1
               release
```

:switch メタコマンド

rig と連携して R のバージョン
を切り替え

履歴・自動補完

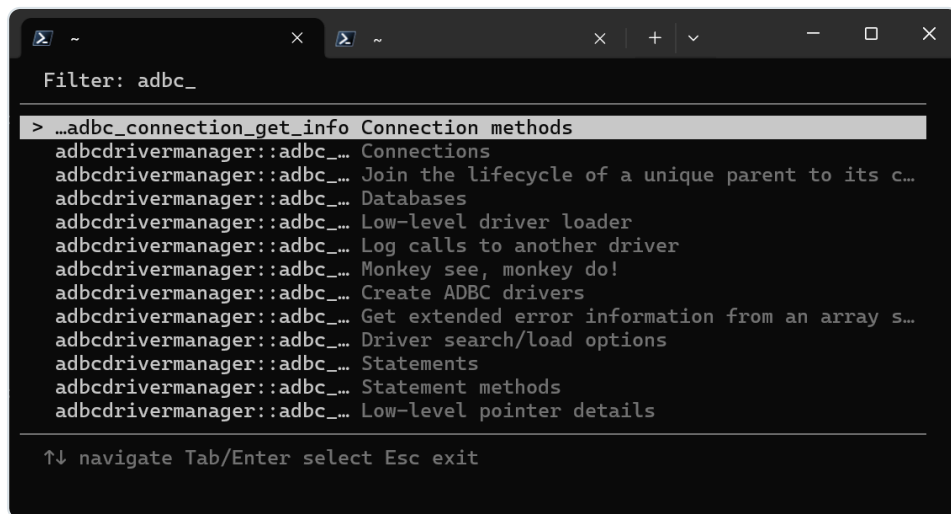
```
R 4.5.2> ? dplyr
0: mtcars |> dplyr::select("cyl")
1: library("dplyr")
2: csv_file <- tempfile(fileext = ".csv")
::write.csv(iris, csv_file, row.names = FALSE)
:::
:::# Create a query plan (LazyFrame) with filtering and group aggregation
:::q <- pl$scan_csv(csv_file)$filter(pl$col("Sepal.Length") > 5)$group_by(
:::....
Page 1: records 0 - 2 total: 3
```

履歴を使った自動補完

- fish 風の履歴補完
- Ctrl+R や履歴ブラウザで履歴をあいまい検索
- 通常の R や radian から履歴を取り込み可能

Help ブラウザ

Atusy 氏の felp パッケージを参考にした機能。ターミナル内でヘルプページを高速検索

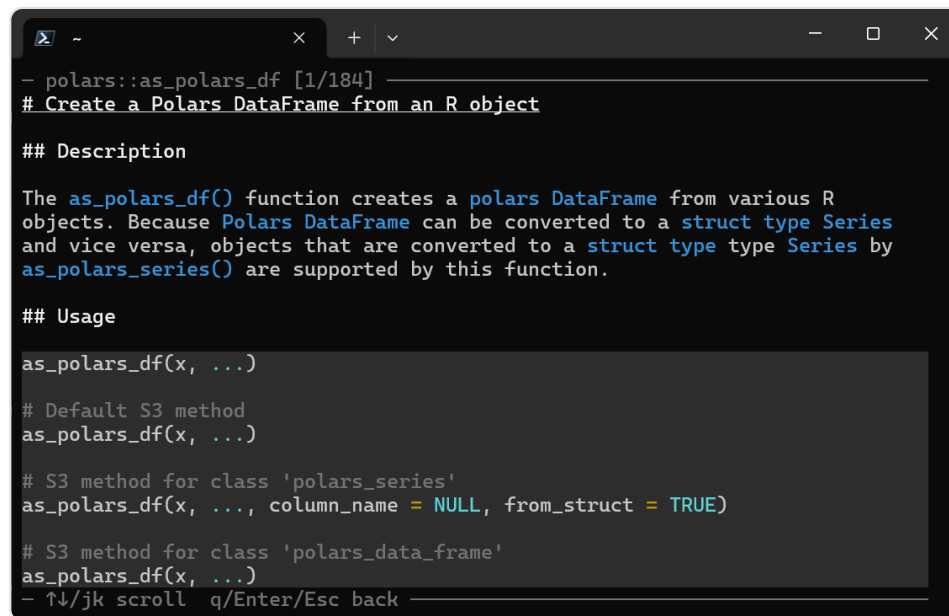


```
Filter: adbc_

> ...adbc_connection_get_info Connection methods
adbcdrivermanager::adbc_... Connections
adbcdrivermanager::adbc_... Join the lifecycle of a unique parent to its c...
adbcdrivermanager::adbc_... Databases
adbcdrivermanager::adbc_... Low-level driver loader
adbcdrivermanager::adbc_... Log calls to another driver
adbcdrivermanager::adbc_... Monkey see, monkey do!
adbcdrivermanager::adbc_... Create ADBC drivers
adbcdrivermanager::adbc_... Get extended error information from an array s...
adbcdrivermanager::adbc_... Driver search/load options
adbcdrivermanager::adbc_... Statements
adbcdrivermanager::adbc_... Statement methods
adbcdrivermanager::adbc_... Low-level pointer details

↑↓ navigate Tab/Enter select Esc exit
```

インストール済パッケージをあいまい検索



```
- polars::as_polars_df [1/184]
# Create a Polars DataFrame from an R object

## Description

The as_polars_df() function creates a polars DataFrame from various R
objects. Because Polars DataFrame can be converted to a struct type Series
and vice versa, objects that are converted to a struct type type Series by
as_polars_series() are supported by this function.

## Usage

as_polars_df(x, ...)

# Default S3 method
as_polars_df(x, ...)

# S3 method for class 'polars_series'
as_polars_df(x, ..., column_name = NULL, from_struct = TRUE)

# S3 method for class 'polars_data_frame'
as_polars_df(x, ...)

- ↑↓/jk scroll q/Enter/Esc back
```

Rd を Markdown に変換して表示

IPC • Headless

外部から R セッション ン进行操作

```
PowerShell 7.6.3
Loading personal and system profiles
took 562ms.
> arf
# arf console v0.4.3
# Edit mode: emacs
# R 4.5.2 is ready.
# Type :cmds for meta commands list,
Ctrl+D to exit.
R 4.5.2> :ipc start
# [arf] IPC server started: \\.\pipe
\arf-ipc-9064
agent> c("右側からこんにちは")
[1] "右側からこんにちは"
R 4.5.2> |

PowerShell 7.6.3
Loading personal and system profiles
took 534ms.
> arf ipc send 'c("右側からこんにちは")'
{
  "accepted": true
}
> arf ipc eval 'nrow(mtcars)'
{
  "error": null,
  "stderr": "",
  "stdout": "",
  "value": "[1] 32"
}
```

- コマンド送信
- コードの評価
- 端末なしで起動
- R 側へパッケージのインストール不要
- IDE / AI 連携を想定
- テスト用途で有用と後から気付く

rd2qmd

Rd ファイルを Markdown・Quarto に変換

- r-polars のウェブサイトで使っている altdoc の Rd から md への変換が不正確なため、表示を直したかった
- Rd を正確に Markdown へ変換する決定的な実装は見つからず
- Rust はこの手の処理に向いているっぽいので、試しに作ってみた

機能紹介

rd2qmd

変換前 / 変換後

Rd

```
\name{simple}  
\title{A Simple Function}  
\usage{simple(x, y = 1)}  
\arguments{  
  \item{x}{The first argument.}  
  \item{y}{Defaults to 1.}  
}  
\examples{simple(1, 2)}
```



Quarto Markdown

```
# A Simple Function  
  
## Usage  
```\r  
simple(x, y = 1)
```\r  
  
## Arguments  
| Argument | Description |  
|---|---|  
| `x` | The first argument. |  
| `y` | Defaults to 1. |
```

使い方

```
rd2qmd convert man/ -o docs/
```

```
# Markdownを出力
```

```
rd2qmd convert file.Rd -f md
```

```
# 並列処理
```

```
rd2qmd convert man/ -o docs/ -j4
```

- 単体の Rd ファイルまたはパッケージ全体を変換
- パッケージ内・外のリンクを解決
- topic index を JSON で生成
- R のインストールは不要

arf への組み込み

rd2qmd

Rd を Markdown へ変換



arf Help Browser

パッケージのヘルプを端末に表示

rd2qmd の変換処理を使って、arf に Help Browser を実装できました。

jgd

JSON を出力する R グラフィックデバイス

httpgd

- vscode 上で R を使う定番
- C++製の描画処理・
HTTP サーバーを同梱
- ビルド大変、CRAN から度々削除

jgd

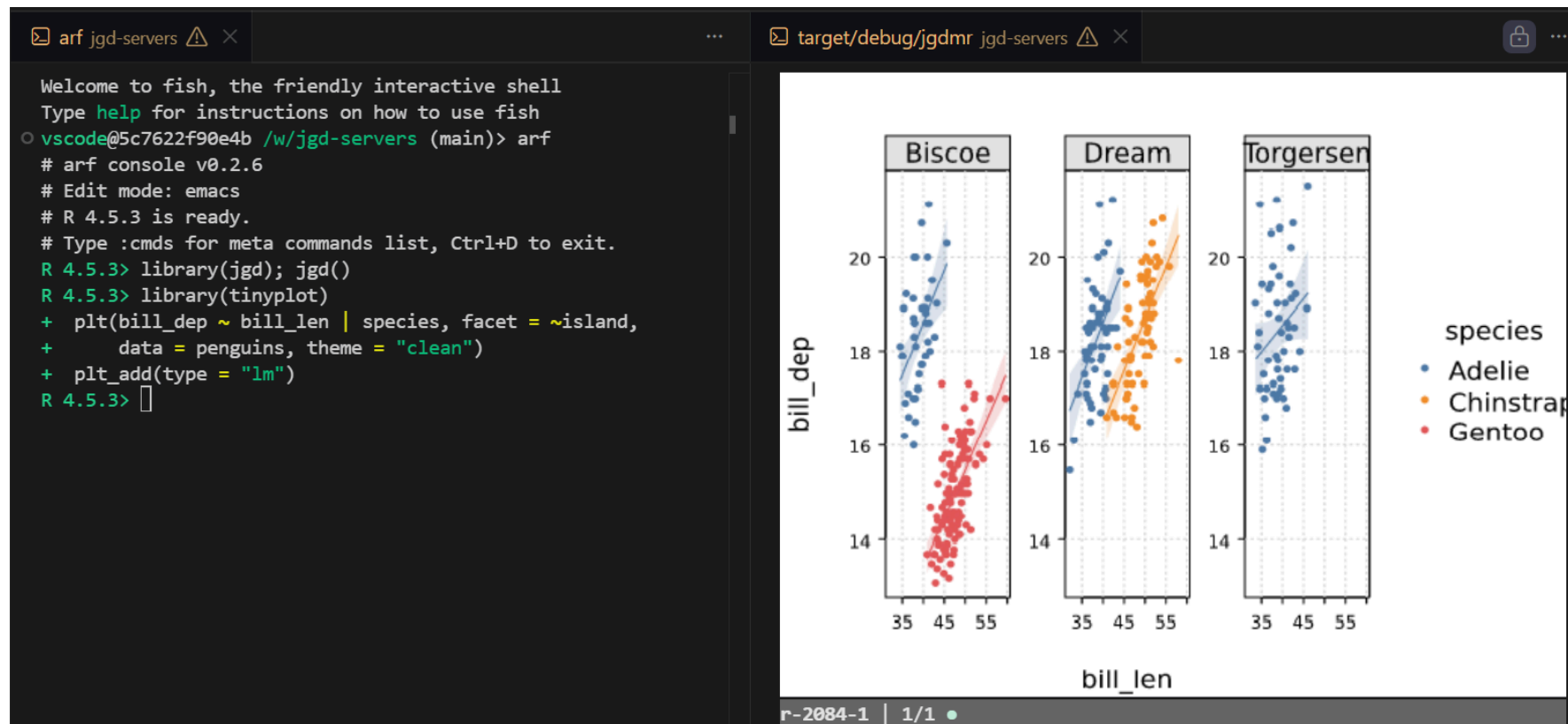
- R パッケージ側は C 製・外部依存なし
- 描画命令を JSON に変換し
外部サーバーへ送信
- JSON の解釈は外部サーバー側の責務

Grant 氏が jgd を公開した際に、「これだ！」と開発に参加

機能紹介

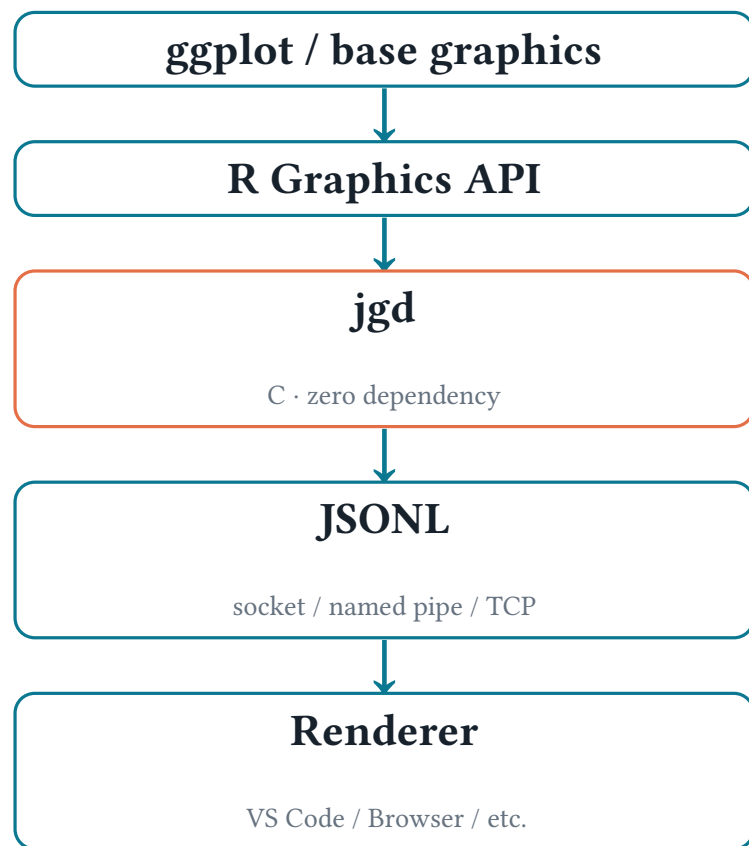
jgd

動作例



画像は開発途中のターミナルグラフィックス対応レンダラー（未公開・飽きて放置中）

アーキテクチャ



- jgd 自身は描画しません
- R の描画命令を記録して JSON へ変換します
- Renderer 側が実際の描画を行います

Renderer

- vscode-r バージョン 3
- Deno 製のブラウザー向けサーバー
- リサイズや plot history にも対応
- jgd パッケージ内の仕様書に沿ってサーバー開発可能

```
install.packages("jgd")
```

```
jgd::jgd()  
plot(cars)
```

jgd の動作確認・E2E テスト

headless arf

R の起動・コード実行

jgd / Renderer

描画命令を連携

headless browser

描画・リサイズ

- ブラウザをリサイズ → R でプロット関数を実行 → 再描画を確認
- コーディングエージェントが R とブラウザの両方を操作・観測でき、開発・デバッグが効率化

まとめ

まとめ

- arf : Rust 製の R コンソール
- rd2qmd : Rd から Markdown・Quarto への変換
- jgd : 描画命令を JSON に変換する R グラフィックデバイス
- 興味があればぜひ使ってみてください

Enjoy!